

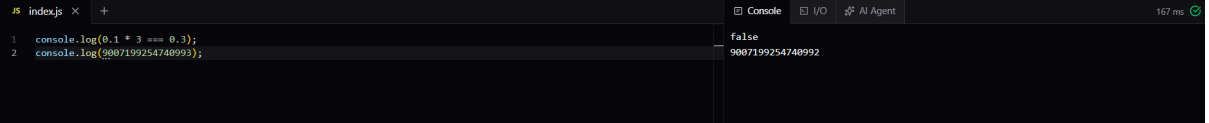
Aluno: Vinicius Ramon Ferreira R.A: 24360978-2

Exercício em duplas (15 min): preveja antes de executar

Para cada trecho: **(a)** qual é a saída? **(b)** o problema é detectado na **compilação**, na **execução** ou **nunca**?

1) JavaScript

a)



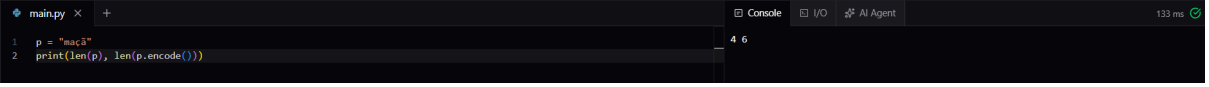
```
1 console.log(0.1 * 3 === 0.3);
2 console.log(9007199254740993);
```

Console: false, 9007199254740992

b) Nunca. Nenhuma das duas linhas gera erro nem aviso. $0.1 * 3$ dá 0.30000000000000004 por causa da representação binária de ponto flutuante (IEEE 754), então a comparação retorna false sem reclamar. O literal 9007199254740993 é maior que `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) e é arredondado silenciosamente para 9007199254740992.

2) Python

a)



```
1 p = "macã"
2 print(len(p), len(p.encode()))
```

Console: 4 6

b) Nunca (na verdade, não há problema). O código é válido e o resultado está correto: `len(p)` conta 4 caracteres (code points) e `len(p.encode())` conta 6 bytes em UTF-8, porque "ç" e "ã" ocupam 2 bytes cada. A diferença só vira bug se alguém confundir número de caracteres com número de bytes, e isso a linguagem não detecta.

3) Go

a)



```
1 package main
2 import "fmt"
3
4 func main() {
5     var b byte = 255
6     b++
7     fmt.Println(b)
8 }
```

Console: 0

b) Nunca. O overflow de inteiro sem sinal em Go é definido pela linguagem e faz wraparound: $255 + 1$ volta para 0, sem erro de compilação nem pânico na execução. (Se fosse uma constante, como `var b byte = 256`, aí o compilador reclamaria. Como `b++` acontece em tempo de execução, passa despercebido.)

4) Java

a)



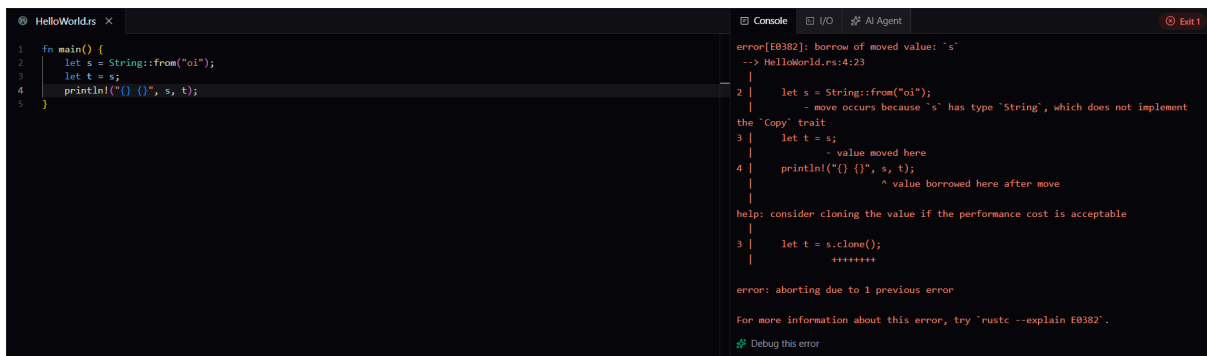
```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int[] v = new int[3];
6         System.out.println(v[0]);
7         System.out.println(v[3]);
8     }
9 }
```

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
at Main.main(Main.java:7)

b) Na execução. O código compila normalmente, pois o compilador não verifica índices de array. Só em tempo de execução a JVM checa os limites e lança `ArrayIndexOutOfBoundsException` em `v[3]`, já que os índices válidos são 0 a 2. Repare que o 0 da linha anterior chegou a ser impresso antes da exceção.

5) Rust

a)



```
1 fn main() {
2     let s = String::from("oi");
3     let t = s;
4     println!("{}", s, t);
5 }
```

error[E0382]: borrow of moved value: `s`
--> HelloWorld.rs:4:23
|
2 | let s = String::from("oi");
| - move occurs because `s` has type `String`, which does not implement the `Copy` trait
3 | let t = s;
| - value moved here
4 | println!("{}", s, t);
| ^ value borrowed here after move
help: consider cloning the value if the performance cost is acceptable
|
3 | let t = s.clone();
| ++++++

b) Na compilação. O borrow checker detecta que `s` teve a posse movida para `t` na linha 3 (String não implementa Copy), então usar `s` na linha 4 é erro E0382. O programa nem chega a ser gerado. Por isso Rust consegue garantir segurança de memória sem coletor de lixo e sem checagem em tempo de execução.

6) C

a)



```
1 #include <stdio.h>
2 int main()
3 {
4     union { int i; float f; } u;
5     u.f = 1.0f;
6     printf("%d\n", u.i);
7 }
```

1065353216

b) Nunca. O código compila e roda sem erro. Ler `u.i` depois de escrever `u.f` reinterpreta os mesmos 4 bytes do float 1.0f (padrão IEEE 754: 0x3F800000) como inteiro, resultando em 1065353216. C não faz checagem de tipo em uniões, e o compilador confia que o programador sabe o que está fazendo. (Em C++ isso seria comportamento indefinido, mas em C99 em diante é permitido.)